



UNIVERZITET U NIŠU
ELEKTRONSKI FAKULTET

TEHNIČKI IZVEŠTAJ

Detekcija i klasifikacija saobraćajnih znakova primenom dubokog učenja

- DUBOKO UČENJE -

Mentor:
Prof. dr Aleksandar Milosavljević

Studenti:
Jovana Malinović 2133
Jovan Paunović 2122

Niš, jun 2026. godina

SADRŽAJ

1. UVOD	3
2. KORIŠĆENE TEHNOLOGIJE	4
3. ARHITEKTURA SOFTVERA.....	5
3.1. Pregled komponenti sistema.....	5
3.2. Komunikacija i tok podataka	5
3.3. Organizacija koda	7
4. DETALJI IMPLEMENTACIJE	8
4.1. Modul za detekciju i praćenje (tracker/sign_tracker.py)	8
4.1.1. Inicijalizacija i konfiguracija.....	8
4.1.2. Filtriranje malih detekcija	8
4.1.3. Logika notifikacije - dva nivoa deduplikacije.....	9
4.1.4. Progresivno poboljšanje isečka	9
4.2. Backend servis (backend/main.py).....	10
4.2.1. Učitavanje modela jednom pri pokretanju	10
4.2.2. WebSocket protokol	10
4.2.3. Blokiranje van event petlje.....	11
4.3. Mobilna aplikacija	11
4.3.1. useDetectionSocket - hook za komunikaciju	11
4.3.2. Sinhronizacija brzine slanja	12
4.3.3. Audio notifikacije bez preklapanja.....	12
5. MODEL I POSTUPAK TRENIRANJA	13
5.1. Izbor modela	13
5.2. Skup podataka.....	13
5.3. Postupak i konfiguracija treniranja.....	14
5.4. Rezultati i evaluacija	15
6. OSTVARENI REZULTATI.....	17
6.1. Pregled ostvarene aplikacije	17
6.2. Rad aplikacije	17
6.3. Ograničenja aplikacije	18
7. ZAKLJUČAK I BUDUĆI RAD	19
LITERATURA	20

1. UVOD

Bezbednost u saobraćaju predstavlja jedan od ključnih globalnih izazova savremenog društva. Prema procenama Svetske zdravstvene organizacije [1], u saobraćajnim nesrećama godišnje strada više od milion ljudi, pri čemu značajan broj tih nesreća nastaje kao posledica neprilagođene brzine ili propuštanja saobraćajnih znakova. Inspiracija za ovaj projekat dolazi iz oblasti sistema za pomoć vozaču (ADAS – Advanced Driver Assistance Systems) [2], kakvi su Tesla Autopilot i Mobileye, koji koriste računarski vid za automatsko prepoznavanje saobraćajne signalizacije. Ovi sistemi su, međutim, zatvoreni i vezani za specifičan hardver. Ovaj projekat polazi od suprotne premise tj. da je funkcionalan sistem moguće izgraditi koristeći isključivo tehnologije otvorenog koda i standardnu mobilnu opremu.

Jezgro sistema čini YOLO detektor fino podešen na TT100K skupu podataka [3], koji u jednom prolazu kroz mrežu istovremeno lokalizuje znak i određuje njegovu klasu. Za praćenje znakova kroz uzastopne frejmove koristi se DeepSORT algoritam, koji svakom detektovanom znaku dodeljuje stabilan identifikator i time sprečava višestruko prijavljivanje istog fizičkog znaka. Sistem prepoznaje 50 klasa saobraćajnih znakova i sastoji se od tri komponente: modula za detekciju i praćenje, FastAPI backend servisa i React Native mobilne aplikacije. Pored vizuelnog prikaza detektovanog znaka, sistem reprodukuje i audio obaveštenje za svaki novootkriveni znak, čime vozač prima informaciju bez skretanja pogleda sa puta. Tri komponente povezane su u klijent-server celinu koja radi u realnom vremenu: mobilna aplikacija šalje kadrove servisu, servis ih prosleđuje modulu za detekciju i praćenje i vraća rezultate, a sva komunikacija odvija se preko jedne WebSocket veze sa niskom latencijom neophodnom za obradu video-toka u hodu. Ostatak izveštaja prati ovu podelu: najpre se opisuju korišćene tehnologije i arhitektura sistema, a zatim detalji implementacije pojedinačnih komponenti, postupak treniranja modela i ostvareni rezultati.

2. KORIŠĆENE TEHNOLOGIJE

YOLO26n (You Only Look Once, Ultralytics) koristi se kao osnova za detekciju i klasifikaciju saobraćajnih znakova. Radi se o jednoprolaznom detektoru koji u jednom prolazu kroz mrežu istovremeno predviđa lokaciju objekta i njegovu klasu, što ga čini pogodnim za primenu u realnom vremenu. Model je fino podešen na TT100K skupu podataka [3] korišćenjem **Kaggle** okruženja koje obezbeđuje besplatan pristup GPU akceleraciji.

DeepSORT (Simple Online and Realtime Tracking with a Deep Association Metric) koristi se za praćenje detektovanih znakova kroz uzastopne frejmove. Algoritam kombinuje Kalmanov filter za predikciju pozicije objekta sa dubokim modelom za ekstrakciju vizuelnih karakteristika, čime se svakom fizičkom znaku dodeljuje stabilan identifikator kroz ceo video tok.

OpenCV se koristi za učitavanje i dekodovanje video frejmova, primenu praga veličine detekcija i vizuelizaciju rezultata.

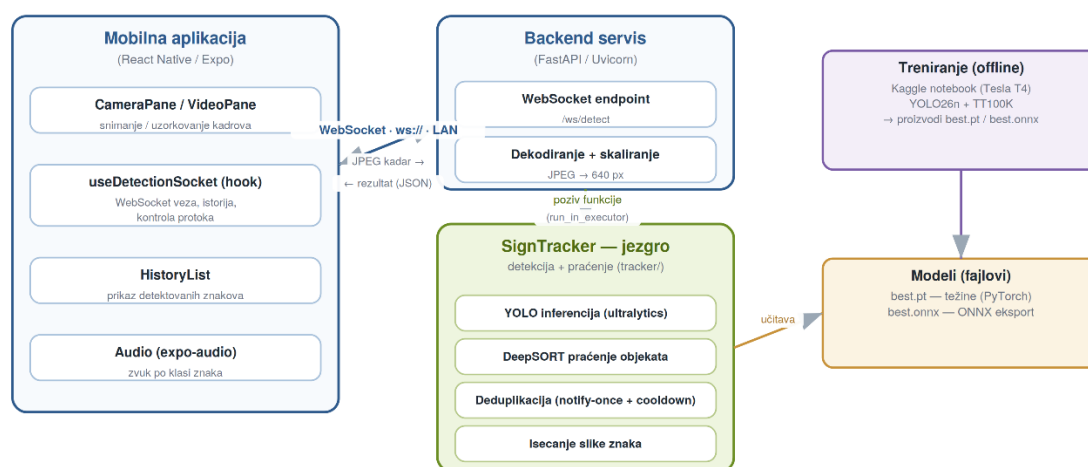
FastAPI je asinhroni Python web framework koji služi kao backend servis sistema. Odabran je zbog nativne podrške za WebSocket konekcije i visokih performansi pri asinhronoj obradi zahteva. WebSocket protokol omogućava dvosmernu komunikaciju između mobilne aplikacije i servera bez troška ponovnog uspostavljanja konekcije pri svakom frejmu.

React Native i **Expo** koriste se za razvoj mobilne aplikacije. React Native omogućava izgradnju nativnih Android aplikacija korišćenjem JavaScript/TypeScript ekosistema, dok Expo pruža alate koji ubrzavaju razvoj i upravljanje zavisnostima. Aplikacija koristi native module za pristup kameri, reprodukciju audio sadržaja i učitavanje video fajlova.

3. ARHITEKTURA SOFTVERA

3.1. Pregled komponenti sistema

Sistem je realizovan kao višekomponentna aplikacija, sastavljena iz tri celine: mobilne aplikacije (klijent), backend-a i jezgra za detekciju i praćenje znakova, u kome se nalazi računarski najzahtevnija logika. Uz njih, model i postupak njegovog treniranja nastaju zasebno od sistema i opisani su u zasebnom poglavlju. Raspored komponenti i veze među njima prikazani su na Slici 1.



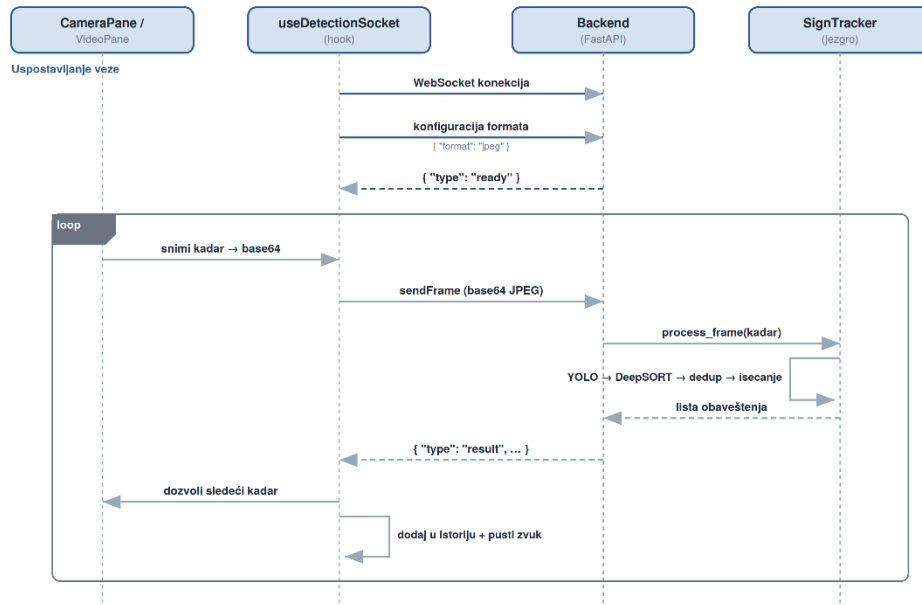
Slika 1. Glavne komponente sistema

Podela odgovornosti jasno je razgraničena među celinama. Jezgro (SignTracker) sadrži svu logiku prepoznavanja i praćenja, kao i odlučivanje o tome kada korisnika treba obavestiti o novom znaku. Projektovano je kao samostalna, ponovo upotrebljiva celina, nezavisna od toga odakle dolaze kadrovi i kome idu rezultati. Backend stoji iznad njega kao jednostavan posrednik: izlaže jezgro preko mreže i povezuje ga sa klijentom, ne preuzimajući sam obradu. Klijentska aplikacija zadužena je za prikaz, pri čemu su vizuelne komponente odvojene od logike komunikacije i upravljanja stanjem. Sve tri celine povezuje jedna WebSocket veza, čiji se tok podataka detaljnije opisuje u nastavku.

3.2. Komunikacija i tok podataka

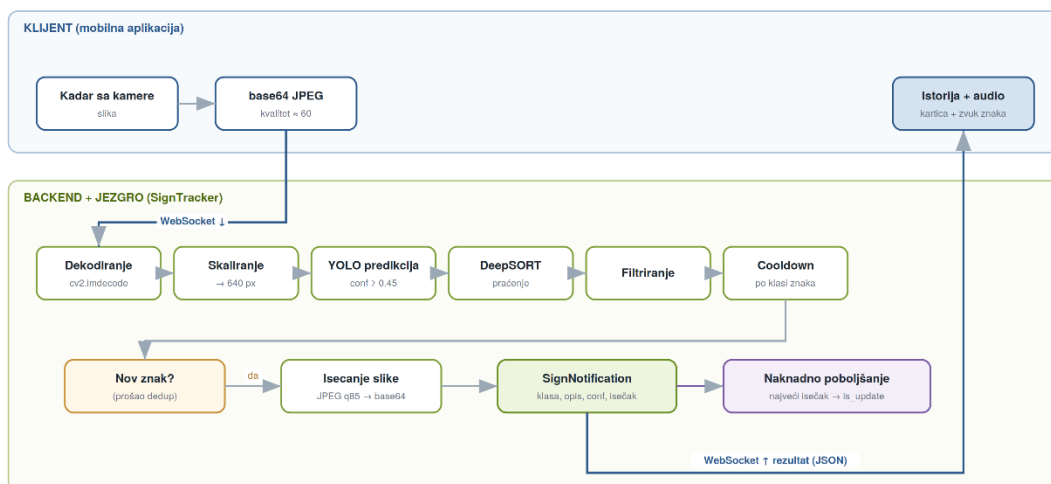
Klijent i server komuniciraju preko jedne *WebSocket* veze. Veza se uspostavlja jednokratnim dogovorom gde klijent pošalje konfiguraciju sa formatom kadrova, a servis potvrdi spremnost porukom “ready”. Nakon toga sledi glavna petlja u kojoj klijent šalje kadrove, a servis za svaki vraća tačno jedan odgovor sa listom detektovanih znakova. Upravo to pravilo “jedan odgovor po kadru” omogućava samoregulaciju protoka gde klijent istovremeno drži ograničen broj kadrova za koje nije stigao odgovor i novi šalje tek kad stigne

prethodni rezultat, pa se brzina slanja sama prilagođava propusnoj moći servisa. Na primer, ako inferencija troši oko 100ms po kadru, klijent će prirodno slati približno deset kadrova u sekundi, bez ikakvog eksplicitnog ograničavanja protoka. Redosled poruka u toku jedne sesije prikazan je na Slici 2.



Slika 2. Sekvencijalni dijagram komunikacije

Dok sekvencijalni dijagram prikazuje ko kome šalje poruke, Slika 3 prikazuje kroz šta svaki kadar prolazi tokom obrade. Na klijentu se snimljeni kadar kodira u JPEG i šalje servisu, gde se dekodira i skalira na rezoluciju koju očekuje detektor. Pripremljeni kadar zatim ulazi u jezgro, gde se redom izvršavaju YOLO detekcija, DeepSORT praćenje, filtriranje i deduplikacija. Tek za znak koji prođe sve te korake iseca se slika i formira obaveštenje. Obaveštenje se serijalizuje i vraća klijentu, koji ga dodaje u istoriju i pušta odgovarajući zvučni signal. Poziv jezgra izvršava se izvan glavne petlje servisa, kako dugotrajna obrada ne bi blokirala primanje novih kadrova.



Slika 3. Tok podataka kroz sistem

3.3. Organizacija koda

Projekat je organizovan tako da svaka glavna komponenta ima svoj direktorijum. Jezgro, servis i mobilna aplikacija fizički su razdvojeni, čime se podela odgovornosti opisana u prethodnim potpoglavljima preslikava i na strukturu izvornog koda:

Projekat/

```
├─ tracker/      # jezgro: detekcija, praćenje i deduplikacija
├─ backend/      # servis (WebSocket)
├─ models/       # datoteke istreniranog modela
├─ notebooks/    # postupak treniranja (offline)
└─ mobile/       # mobilna aplikacija (React Native / Expo)
    ├─ app/       # ekrani aplikacije
    ├─ components/ # vizuelne komponente (video prikaz, lista znakova)
    ├─ lib/       # hook za komunikaciju sa servisom
    └─ assets/    # zvučni signali po klasi znaka
```

Mobilna aplikacija interno prati istu logiku razdvajanja: ekrani i vizuelne komponente odvojeni su od logike vezane za komunikaciju, koja je smeštena u poseban direktorijum.

4. DETALJI IMPLEMENTACIJE

4.1. Modul za detekciju i praćenje (tracker/sign_tracker.py)

Klasa SignTracker predstavlja srž sistema. Projektovana je bez pretpostavki o izvoru ulaza. Ona prima kadar kao niz piksela i vraća listu obaveštenja, čime ostaje samostalna i ponovo upotrebljiva i u standalone režimu i kao pozadinska logika backend servisa.

4.1.1. Inicijalizacija i konfiguracija

Pri instanciranju se učitava YOLO model i gradi DeepSORT modul za praćenje. Svi parametri praćenja koji se podešavaju čitaju se iz deepsort.yaml, što znači da se ponašanje sistema može podešavati bez izmene koda:

```
with open(tracker_config, "r") as f:
    cfg = yaml.safe_load(f)

self._deepsort_kwargs = dict(
    max_age=cfg.get("max_age", 60),
    n_init=cfg.get("n_init", 3),
    max_cosine_distance=cfg.get("max_cosine_distance", 0.4),
    ...
)
self.deepsort = DeepSort(**self._deepsort_kwargs)
```

Parametar `n_init` određuje koliko uzastopnih frejmova detekcija mora biti prisutna pre nego što DeepSORT “potvrdi” praćenje. Ovo eliminiše lažne detekcije koje se pojave samo u jednom ili dva frejma. Parametar `max_age` definiše koliko frejmova DeepSORT čuva trag u odsustvu detekcije pre nego što ga ugasi.

4.1.2. Filtriranje malih detekcija

Pre nego što detekcije stignu do praćenja, svaka koja je ispod praga `min_box_size` se odbacuje:

```
box_side = min(x2 - x1, y2 - y1)
if self.min_box_size and box_side < self.min_box_size:
    continue
```

Ovo je ključna odluka dizajna jer YOLO često detektuje znak sa visokom pouzdanošću čak i kada je još uvek daleko i ima svega nekoliko piksela. Takva detekcija bi bila klasifikovana i prijavljena vozaču pre nego što je znak uopšte čitljiv, što može dovesti do pogrešne klasifikacije. Filterom se osigurava da se znak prijavi tek kada je dovoljno blizu da se može pouzdano klasifikovati.

4.1.3. Logika notifikacije – dva nivoa deduplikacije

Problem koji sistem mora da reši je sledeći: isti fizički znak prolazi kroz desetine ili stotine frejmova i bez deduplikacije, sistem bi vozaču reprodukovao isti zvuk i prikazao isti znak više puta.

Prvi nivo je deduplikacija po track – svako praćenje se prijavljuje tačno jednom:

```
if track_id not in self._notified_ids:
    self._notified_ids.add(track_id)
    ...
```

Međutim, DeepSORT može privremeno izgubiti trag (zbog zaklanjanja, zamućenja pri kretanju ili kratkog izlaska znaka iz kadra) i zatim ga ponovo detektovati pod novim track_id. Bez drugog nivoa zaštite, to bi izazvalo duplikat. Drugi nivo je class-level cooldown:

```
recently_active = (
    class_name in self._last_class_activity
    and now - self._last_class_activity[class_name] <
self.class_cooldown_seconds
)

if recently_active:
    # ista klasa je bila aktivna u poslednjih N sekundi - potisni duplikat
    pass
else:
    notification = self._make_notification(t, frame, frame_w, frame_h)
    notifications.append(notification)
```

Svaki put kada se detektuje potvrđeni znak bilo koje klase, vreme poslednje aktivnosti te klase se osvežava. Ako ista klasa ponovo postane aktivna unutar prozora class_cooldown_seconds, notifikacija se potiskuje. Dakle novi znak iste klase koji se pojavi nakon dužeg odsustva (duže od cooldown prozora) normalno prolazi.

4.1.4. Progresivno poboljšanje isečka

Inicijalno obaveštenje šalje se dok je znak još daleko. Bounding box je mali i isečak je mutan. Sistem kontinuirano prati koji je isečak do sada imao najveću površinu i samim tim je najčešće najjasniji:

```
if area > es.best_area:
    new_crop = self._crop_sign(frame, (x1, y1, x2, y2))
    if new_crop is not None:
        es.best_area = area
        es.best_image_b64 = new_crop
        es.improved = True
```

Kada znak napusti kadar (trag nije viđen DISPLAY_TTL frejmova), sistem šalje klijentu ažuriran isečak kao posebnu is_update poruku koja nosi isti track_id i timestamp kao originalno obaveštenje. Klijent na osnovu toga može da zameni prvobitnu, zamagljenu sliku oštrijim prikazom bez dodavanja novog reda u listu.

4.2. Backend servis (backend/main.py)

Backend je tanak sloj iznad SignTracker-a: prima kadrove sa mreže, priprema ih i serijalizuje rezultate ka klijentu. Implementiran je kao FastAPI aplikacija sa jednom WebSocket rutom.

4.2.1. Učitavanje modela jednom pri pokretanju

YOLO i DeepSORT embedder su skupi za inicijalizaciju (nekoliko sekundi i stotine MB memorije). Korišćenjem FastAPI lifespan kontekst menadžera, model se učitava jednom pri pokretanju servisa, a ne pri svakoj konekciji:

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    global _tracker
    _tracker = SignTracker(model_path=str(MODEL_PATH), ...)
    yield
    print("[backend] Shutting down.")
```

Za novu sesiju nije potrebno ponovo učitavati model. Dovoljno je pozvati `_tracker.reset()` koji briše interno stanje (aktivna praćenja, skup prijavljenih ID-ova, cooldown stanje) bez ponovnog učitavanja težina.

4.2.2. WebSocket protokol

Protokol je dvofazni. U fazi rukovanja, klijent šalje JSON konfiguraciju i server odgovara sa ready:

```
cfg = await ws.receive_json()
fmt = str(cfg.get("format", "raw")).lower()
_tracker.reset()
await ws.send_json({"type": "ready", "format": fmt})
```

Nakon toga počinje striming petlja u kojoj važi pravilo “jedan frejm → jedan odgovor”. Na strani servera to znači da se odgovor šalje na svaki primljeni kadar, čak i kada je neispravan, pa klijent uvek dobije povratnu informaciju i petlja se ne zaglavљуje:

```
if frame is None:
    await ws.send_json({"type": "result", "frame": ..., "detections": [],
"warning": warning})
    continue

notifications = await loop.run_in_executor(None, _tracker.process_frame,
...)
await ws.send_json({"type": "result", "detections": [...]}))
```

4.2.3. Blokiranje van event petlje

YOLO inferencija i DeepSORT su sinhroni, CPU/GPU-intenzivni pozivi koji bi blokirali asinhroni server i onemogućili primanje poruka tokom obrade:

```
notifications = await loop.run_in_executor(
    None, _tracker.process_frame, frame, frame_w, frame_h
)
```

`run_in_executor` prebacuje blokirajuće funkcije u thread pool, dok event petlja ostaje slobodna za upravljanje konekcijom.

4.3. Mobilna aplikacija

Aplikacija je organizovana po principu jasnog razdvajanja odgovornosti: komunikaciona logika enkapsulirana je u hook, vizuelne komponente su odvojene po funkciji, a glavni ekran samo spaja delove u celinu.

4.3.1. useDetectionSocket – hook za komunikaciju

Sva WebSocket logika živi u jednom hook-u koji izlaže minimalan API ka komponentama: stanje konekcije, istoriju detekcija, broj poslatih frejmova i funkcije connect, disconnect, sendFrame, clearHistory.

Konekcija prolazi kroz tačno definisane faze. Pri otvaranju veze, klijent odmah šalje konfiguracionu poruku:

```
ws.onopen = () => {
  ws.send(JSON.stringify({ format: 'jpeg' }));
};
```

Tek kada server odgovori sa “ready”, `readyRef.current` se postavlja na “true” i frejmovi mogu da se šalju. Ovo sprečava slanje frejmova pre nego što je server spreman.

Dolazne poruke se razlikuju po tipu. Novi znakovi se dodaju na početak liste (najnoviji na vrhu), dok `is_update` poruke pronalaze postojeći zapis i zamenjuju mu sliku:

```
const updates = dets.filter((d) => d.is_update);
const fresh = dets.filter((d) => !d.is_update);

setHistory((prev) => {
  let next = prev;
  if (updates.length) {
    next = next.map((item) => {
      const u = updates.find(
        (d) => d.track_id === item.track_id && d.timestamp ===
        item.timestamp
      );
      if (u) {
        return { ...item, image: u.image };
      }
      return item;
    });
  }
  return [...updates, ...next];
});
```

```

    );
    return u ? { ...item, image_b64: u.image_b64 } : item;
  });
}
return fresh.length ? [...[...fresh].reverse(), ...next] : next;
});

```

4.3.2. Sinhronizacija brzine slanja

Aplikacija ne sme da šalje novi frejm pre nego što server odgovori na prethodni. Umesto globalnog stanja, ovaj uslov se implementira kroz callback koji komponente registruju pri montiranju:

```

// u index.tsx
<CameraPane
  registerOnResult={({cb} => { onResultRef.current = cb; }}
  ...
/>

// u useDetectionSocket-u, pri svakom odgovoru servera:
onResultRef.current?.();

```

Kada CameraPane ili VideoPane primi ovaj callback poziv, otključava slanje sledećeg frejma, koje je do tada bilo zadržano. Ovako se pacing implementira lokalno u svakoj komponenti bez potrebe za deljenim stanjem ili timer-ima.

4.3.3. Audio notifikacije bez preklapanja

Svaki novi znak dodaje se u red čekanja. Zvukovi se ne reprodukuju konkurentno već svaki čeka da prethodni završi:

```

const playNext = useCallback(() => {
  if (isPlayingRef.current) return; // već se nešto reprodukuje
  const source = queueRef.current.shift();
  if (!source) return;

  isPlayingRef.current = true;
  const p = createAudioPlayer(source);
  p.addListener('playbackStatusUpdate', (status) => {
    if (!status.didJustFinish) return;
    isPlayingRef.current = false;
    playNext(); // nastavi sa sledećim u redu
  });
  p.play();
}, []);

```

Ovo osigurava da vozač čuje svako obaveštenje redom, bez gušenja zvukova kada se u kratkom vremenskom intervalu detektuje više znakova.

5. MODEL I POSTUPAK TRENIRANJA

5.1. Izbor modela

Za detekciju saobraćajnih znakova izabran je model **YOLO26** u nano varijanti (yolo26n). YOLO26 je najnovija generacija iz familije YOLO modela u okviru Ultralytics biblioteke i, kao takva, predstavlja preporučeni izbor sa zvanično dokumentovanim postupkom za dotreniravanje na sopstvenom skupu podataka [4]. Korišćenje aktuelne generacije obezbeđuje pristup najnovijim unapređenjima u tačnosti i brzini, kao i aktivnoj podršci kroz dokumentaciju.

Od ponuđenih veličina modela odabrana je nano varijanta, najmanja i najbrža u familiji. Ovaj izbor proističe iz zahteva za radom u realnom vremenu: tokom razvoja aplikacije inferencija se izvršavala lokalno na laptop računaru, bez namenske grafičke kartice, pa je nisko vreme inferencije bilo presudno za održavanje obrade video-toka u realnom vremenu. U takvim uslovima brzina nano modela bitnija je od poslednjih nekoliko procenata tačnosti koje bi pružio veći, ali sporiji model.

Model nije treniran od nule, već je primenjeno **preneseno učenje**. Pošlo se od unapred istreniranih težina (yolo26n.pt), koje su zatim dotrenirane na skupu saobraćajnih znakova. Time se iskorišćava opšte znanje koje je model već stekao na velikom skupu slika, pa je za prilagođavanje konkretnom zadatku potrebno znatno manje podataka i vremena nego pri treniranju od nule.

5.2. Skup podataka

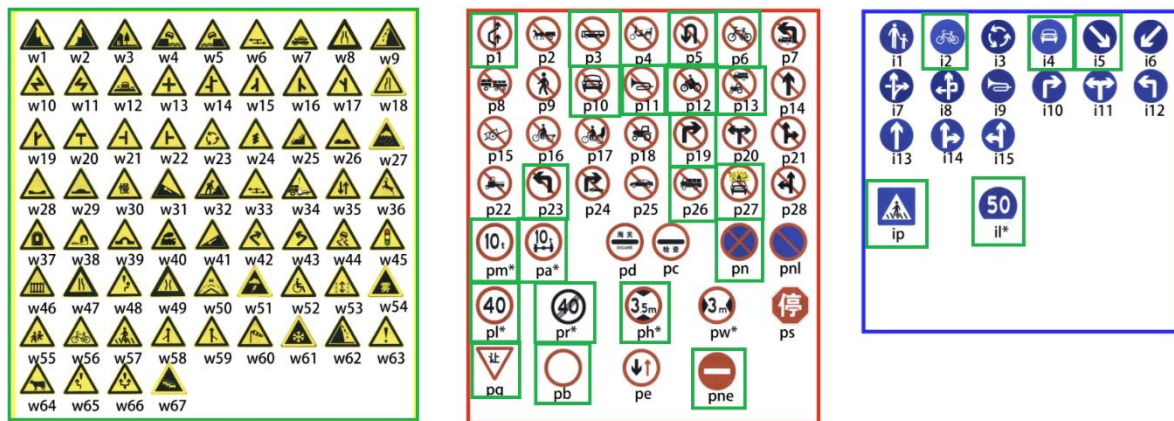
Za treniranje je korišćen skup podataka **TT100K** (Tsinghua-Tencent 100K) dostupan na [5], koji predstavlja kolekciju stvarnih panoramskih fotografija kineskih ulica snimljenih iz vozila. Skup je naročito pogodan za ovaj zadatak jer sadrži znakove u realnim uslovima vožnje, različite udaljenosti, osvetljenja i zaklonjenosti – što odgovara uslovima u kojima aplikacija treba da radi.

Od ukupnog skupa korišćen je podskup od **50 najčešćih klasa** znakova, koje su označene na Slici 4. Oznake klasa su kodirane prema sistemu iz TT100K skupa, gde slovni prefiks označava kategoriju znaka, a sufiks dodatno značenje – na primer, *pl80* označava ograničenje brzine na 80 km/h, prefiks *w* znak upozorenja, *i* informativni ili obavezni znak, a *il* najmanju dozvoljenu brzinu. Validacioni skup sadrži 3.627 slika sa ukupno 10.823 obeleženih instanci znakova.

Anotacije su date u YOLO formatu: uz svaku sliku ide tekstualna datoteka u kojoj svaki red opisuje po jedan znak vrednostima klasa *cx cy w h*, gde su koordinate centra i dimenzije okvira normalizovane na opseg od 0 do 1 u odnosu na dimenzije slike. Putanje do slika i spisak

```
train: { /kaggle/input/datasets/braunge/tt100k/mydata}/images/train
val: { /kaggle/input/datasets/braunge/tt100k/mydata}/images/val

nc: 50
names:
['pl80', 'p6', 'ph', 'w', 'pa', 'p27', 'i5', 'p1', 'il70', 'p5', 'pm', 'p19', 'ip', 'p11', 'p13', 'p26', 'i2', 'pn', 'p10', 'p23', 'pbp', 'p3', 'p12', 'pne', 'i4', 'pb', 'pg', 'pr', 'pl5', 'pl10', 'pl15', 'pl20', 'pl25', 'pl30', 'pl35', 'pl40', 'pl50', 'pl60', 'pl65', 'pl70', 'pl90', 'pl100', 'pl110', 'pl120', 'il50', 'il60', 'il80', 'il90', 'il100', 'il110']
```



5.3. Postupak i konfiguracija treniranja

Tabela 1. Vrednosti hiperparametara pri treniranju modela

<i>Parametar</i>	<i>Vrednost</i>
<i>Broj epoha</i>	30
<i>Rano zaustavljanje (patience)</i>	8 epoha
<i>Rezolucija slike</i>	640×640

Mehanizam ranog zaustavljanja prekida treniranje ako tačnost ne napreduje tokom osam uzastopnih epoha, čime se sprečava nepotrebno trošenje resursa i prenaučavanje (engl. *overfitting*). Po završetku treniranja, težine najboljeg modela sačuvane su u datoteci *best.pt*, koja je zatim izvezena i u ONNX format radi mogućnosti pokretanja u različitim okruženjima za inferenciju.

5.4. Rezultati i evaluacija

Nakon treniranja, najbolji model evaluiran je na validacionom skupu od 3.627 slika sa 10.823 instance znakova. Ukupne metrike prikazane su u Tabeli 2. Treba napomenuti da isti validacioni skup služi i za izbor najboljeg modela tokom treniranja i za prikazane konačne metrike. Zaseban test skup nije izdvojen, pa prikazane vrednosti mogu blago precenjivati stvarnu tačnost na potpuno neviđenim podacima.

Tabela 2. Ukupne metrike modela na validacionom skupu

Metrika	Vrednost
<i>mAP@0.5</i>	0,566
<i>mAP@0.5:0.95</i>	0,458
<i>Preciznost (precision)</i>	0,602
<i>Odziv (recall)</i>	0,509

Posmatrano po klasama, uspešnost modela znatno varira. Najbolje rezultate model postiže na vizuelno izraženim i međusobno različitim znakovima – na primer, klase *pr*, *pne*, *i5* i *pn* dostižu *mAP@0.5* između 0,87 i 0,96. Najslabiji rezultati javljaju se kod znakova ograničenja brzine koji se međusobno razlikuju samo po ispisanom broju, poput klasa *pl30* (0,21), *pl65* (0,25) i *pl20* (0,32). Ovakav raspon je očekivan: znakovi ograničenja brzine vizuelno su gotovo identični – isti oblik, boja i okvir, uz razliku jedino u cifri – pa ih model često međusobno meša, naročito kada su udaljeni ili mutni. Ovo ujedno predstavlja i glavno ograničenje tačnosti sistema, o čemu će biti reči u poglavlju o rezultatima.

Primeri rada modela na slikama iz validacionog skupa prikazani su na Slikama 5 i 6. Na Slici 5 model uspešno detektuje dva znaka sa visokom pouzdanošću, dok Slika 6 prikazuje detekciju znaka ograničenja brzine u tipičnoj sceni vožnje na autoputu.



Slika 5. Primer detekcije sa visokom pouzdanošću

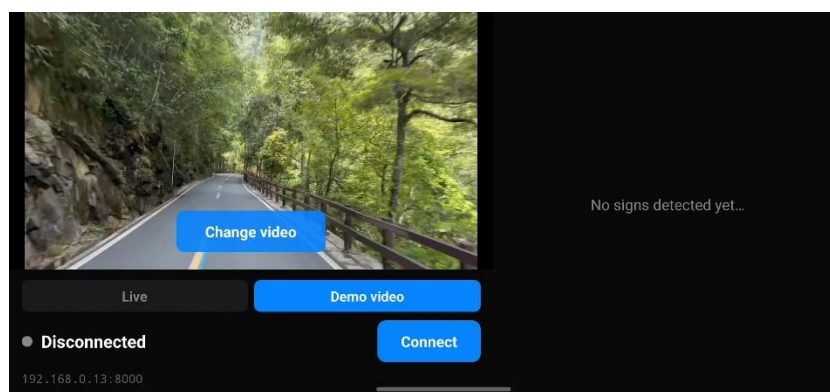


Slika 6. Detekcija znaka u sceni vožnje

6. OSTVARENI REZULTATI

6.1. Pregled ostvarene aplikacije

Krajnji rezultat projekta je funkcionalna mobilna aplikacija koja, u sprezi sa razvijenim servisom, u realnom vremenu prepoznaje saobraćajne znakove i obaveštava korisnika o svakom novom znaku. Opšti izgled aplikacije prikazan je na Slici 7. Ekran je podeljen na dva dela: levi panel sadrži video-prikaz sa kontrolama, a desni panel prikazuje istoriju detektovanih znakova.



Slika 7. Izgled aplikacije

Aplikacija podržava dva režima rada. U režimu uživo (Live) obrađuju se kadrovi sa kamere uređaja, dok režim demo prikaza (Demo video) omogućava pokretanje detekcije nad unapred snimljenim video-zapisom, što je korisno za testiranje i demonstraciju. Povezivanje sa servisom obavlja se dugmetom Connect, pri čemu je u donjem delu ekrana prikazan indikator stanja veze (na primer, Disconnected pre povezivanja, odnosno Streaming tokom aktivne obrade). Dok detekcija ne počne, panel istorije prikazuje poruku da nijedan znak još nije detektovan.

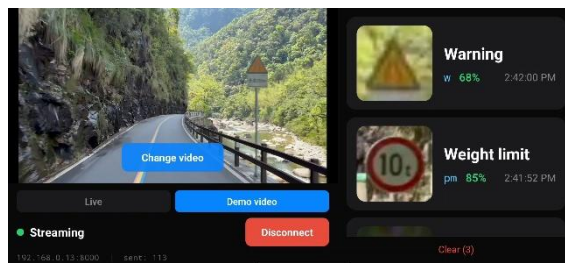
6.2. Rad aplikacije

Tokom obrade video-toka, svaki prepoznati znak prikazuje se kao zasebna kartica u panelu istorije. Kartica sadrži isečenu sliku stvarnog znaka iz kadra, čitljiv naziv znaka (na primer “Weight limit” ili “Warning”), oznaku klase, procenat pouzdanosti detekcije i vreme kada je znak prepoznat. Korišćenje isečka stvarnog znaka, umesto unapred pripremljene ikonice, omogućava korisniku da i sam proceni o kom se znaku radi. Svaku novu detekciju prati i zvučni signal u vidu glasa koji izgovara koji znak je detektovan, čime se korisnik obaveštava bez potrebe da gleda u ekran.

Detekcije se u istoriji nižu hronološki, sa najnovijim znakom na vrhu, pri čemu se svaki fizički znak prijavljuje samo jednom. Slike 8 i 9 prikazuju isti tok rada u dva različita trenutka: na Slici 8 istorija sadrži dve detekcije, dok je na Slici 9, snimljenoj nešto kasnije, pridodata i treća. Time se ilustruje kako se lista detektovanih znakova postepeno popunjava kako vožnja odmiče, zadržavajući pregled svih do tada prepoznatih znakova.



Slika 8. Detekcija znakova tokom rada



Slika 9. Rast istorije detekcija kroz vreme

6.3. Ograničenja aplikacije

Iako sistem uspešno prepoznaje znakove u realnom vremenu, u radu postoje određena ograničenja. Sistem povremeno propusti znak ili ga ponekad pogrešno klasifikuje – najčešće kod vizuelno sličnih znakova koji se razlikuju samo u sitnim detaljima (na primer, znakovi ograničenja brzine). Ove pojave prevashodno proističu iz ograničene tačnosti korišćenog nano modela, o čemu je bilo reči u poglavlju o modelu, pa bi se najvećim delom mogle ublažiti upotrebom većeg i tačnijeg modela, po cenu većih hardverskih zahteva.

Dodatno ograničenje predstavlja skup podataka na kome je model treniran. Demonstracija je rađena nad kineskim saobraćajnim znakovima iz skupa TT100K, pa bi za pouzdanu primenu na domaćim putevima bilo potrebno model dotrenirati na skupu znakova koji odgovaraju lokalnoj signalizaciji. Takođe, sistem je razvijen i testiran kao prototip u lokalnoj mreži, tako da bi za stvarnu upotrebu bili neophodni dalji razvoj i optimizacija.

7. ZAKLJUČAK I BUDUĆI RAD

U okviru ovog projekta razvijen je sistem za detekciju saobraćajnih znakova u realnom vremenu, zasnovan na mobilnoj aplikaciji i serveru koji koristi dotrenirani model dubokog učenja. Sistem uspešno objedinjuje detekciju zasnovanu na modelu YOLO26n, praćenje objekata algoritmom DeepSORT i logiku koja obezbeđuje da se svaki znak korisniku prijavi tačno jednom, uz prikaz isečka stvarnog znaka, opisa i pouzdanosti, kao i propratni zvučni signal. Time su ostvareni svi postavljeni ciljevi: prepoznavanje znakova u realnom vremenu i pregledno obaveštavanje vozača.

Postignuti rezultati pokazuju da je pristup upotrebljiv, ali i da postoji prostor za dalja unapređenja, pre svega u pravcu prevazilaženja ograničenja navedenih u prethodnom poglavlju. Najveći doprinos tačnosti doneo bi prelazak na veći i precizniji model, dok bi za primenu na domaćim putevima model trebalo dotrenirati na lokalnoj signalizaciji. Pored toga, skup klasa mogao bi se proširiti izvan trenutnih 50, a sam sistem, zasad realizovan kao prototip, dalje razviti u pravcu rada na uređaju bez stalne veze sa serverom i istovremene upotrebe od strane više korisnika.

LITERATURA

- [1] W. H. O. WHO, 2023. [Online]. Available:
<https://www.who.int/publications/i/item/9789240086517>.
- [2] Ekim Yurtsever, Jacob Lambert, Alexander Carbello and Kazuya Takeda, "A Survey of Autonomous Driving: Common Practices and Emerging Technologies," 2020.
- [3] [Online]. Available: <https://docs.ultralytics.com/datasets/detect/tt100k#key-features>.
- [4] "Link to this sectionHow to Fine-Tune YOLO on a Custom Dataset," Ultralytics, April 2026. [Online]. Available: <https://docs.ultralytics.com/guides/finetuning-guide#fine-tuning-vs-training-from-scratch>. [Accessed 24 June 2026].
- [5] "TT100K," Kaggle, [Online]. Available:
<https://www.kaggle.com/datasets/braunge/tt100k>. [Accessed 24 June 2026].